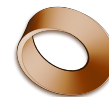


Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science



Year 11 2026

Programming 5 — String Activities — ANSWERS

Work down the page. You are not expected to reach the bottom.

Warm up

1 • Predict, then run adapted from Paul Long

Two strings for this one:

```
a = "Mississippi"
b = "Count every character!"
```

Write your prediction *before* you type anything.

ERL	Prediction	Actual
a.length		11
b.length		22
a.substring(0,4)		Miss
b.left(5)		Count
b.right(10)		character!
b.substring(6,5)		every

Count the spaces. They are characters too, and the exclamation mark counts as one as well.

Build

2 • Initials

Ask the user for their full name and print their initials.

Ada Augusta Lovelace should give AAL.

Start with two names and get it working, then worry about people with three.

Mark scheme

Worked solution — initials. Tested, CPython 3.9.6.

```
name = input("Full name: ")
initials = ""
for word in name.split():          # split() breaks it at the spaces
    initials = initials + word[0]
print(initials.upper())
```

Tested: Ada Augusta Lovelace → AAL · Grace Hopper → GH · Alex Rawson → AR.

`split()` **is new and worth naming**. It cuts a string into a list at every space. It is not on the J277 list of string operations, so **do not require it** — a student who asks for the first name and surname *separately* and takes `first[0] + last[0]` has solved the set task completely and correctly.

That simpler version is the expected answer:

```
first = input("First name: ")
last  = input("Surname: ")
print((first[0] + last[0]).upper())
```

The loop version is the extension, and the interesting difference is that it copes with any number of names without being told how many — a running total again, but building a string instead of adding numbers.

3 · The race code adapted from Paul Long — the good one

A swimming gala gives every race a short code, built out of four pieces of information.

Variable	Value	What goes into the code
gender	"Boys"	the first 1 character
distance	"400"	all of it
event	"Relay"	the first 2 characters
age_group	"Under 13"	the first 1 , then the last 2

Work out the code on paper first. Then write the program that builds it.

The code is: **B400ReU13**

Then make it useful: ask the user for the four pieces rather than fixing them in the code, so it works for any race in the gala.

Check yourself on age_group. It is "Under 13" — **8** characters, and the space is one of them. You need the **first 1** and the **last 2**, and nothing in between. **So U and 13, giving U13. Taking substring(0,1) and right(2) is the clean way; counting to position 6 by hand works but breaks the moment the age group changes.**

4 • The part number adapted from Paul Long

A factory builds a part number from three things:

```
manuf_year = "2010"
manuf_name = "Blobby Enterprises"
stock_code = "0987654321"
```

The part number is **the last digit of the year**, then **the first 2 characters of the name**, then **the last 3 digits of the stock code**.

The part number is: **0Bl321**

Note the capital B and the lower-case l. You are taking characters exactly as they are — no one asked you to change the case.

5 • Password checker

Ask the user for a password and tell them whether it is acceptable. It must be:

- at least 8 characters long
- **not** the word password, in any combination of upper and lower case

*The second rule is the interesting one. PassWord and PASSWORD must both be rejected — and you are not going to list every combination. **Convert what they typed to lower case first, then compare once.***

Mark scheme

Worked solution — password checker. Tested, CPython 3.9.6.

```
pw = input("Choose a password: ")

if len(pw) < 8:
    print("Too short")
elif pw.lower() == "password":
    print("Too obvious")
else:
    print("Accepted")
```

Tested: abc → too short · password, PassWord and PASSWORD → all rejected · correcthorse and Password1 → accepted.

The whole lesson is in `pw.lower() == "password"`. There are 256 ways to capitalise those eight letters. Converting to lower case first collapses all of them into one comparison — **this is what case conversion is FOR**, and it is a far better argument for it than “it makes text tidy”.

Marking: a student listing several spellings with or has understood the problem and not the tool. Give the marks, then show them the one-line version.

Password1 being accepted is worth thirty seconds of discussion — our rule only bans the exact word, and real password checks are harder than they look.

Stretch

6 • The Caesar shift character codes doing real work

Julius Caesar encrypted his messages by moving every letter three places along the alphabet. A becomes D, B becomes E, and so on.

HELLO becomes KHOOR.

You already have everything you need: ASC turns a character into its number, CHR turns a number back into a character. Adding 3 in between is the whole cipher.

- (a) Encrypt a single letter typed by the user.
- (b) Encrypt a whole word. *You will need a loop.*
- (c) Decrypt it again, and check you get the original back.

The problem you will hit. What happens to Z? Adding 3 takes you past the end of the alphabet into punctuation.

It has to wrap round — Z should become C. And the operator that wraps things round is MOD, which you met in Programming 4. MOD 26, because there are 26 letters.

Mark scheme

Worked solution — the Caesar shift. Run and tested, CPython 3.9.6.

```
text = input("Message (capitals): ")
key = int(input("Shift by: "))

result = ""
for ch in text:
    if ch >= "A" and ch <= "Z":
        position = ord(ch) - 65          # A becomes 0, Z becomes 25
        position = (position + key) % 26  # add, and wrap round
        result = result + chr(position + 65) # back to a letter
    else:
        result = result + ch             # spaces etc. pass through
print(result)
```

Tested: HELLO +3 → KHOOR · ZEBRA +3 → CHEUD · ATTACK AT DAWN +5 → FYYFHP FY IFBS. Running each result back through with a **negative** key returns the original — that is part (c), and it needs no new code.

The three lines that matter, and why.

1. - 65 then + 65. You cannot take MOD 26 of 72. Subtracting 65 slides A–Z down to 0–25, where wrapping makes sense; adding it back afterwards returns you to letters. **Students who skip this get punctuation out of Z.**
2. % 26 is the wrap. ZEBRA is the test case — if Z comes out as C, the wrap works.
3. The else is not optional. Without it, spaces are shifted too and the message becomes unreadable in a way that looks like a bug in the maths.

Note this is the only solution in the pack that needs a loop over a string — it is genuinely a stretch task, and worth doing at the board if nobody gets there alone.

One last thing, and then we will never mention it again. **A telephone number is a String.** Store 01382 as an Integer and the leading zero vanishes, because an Integer has no idea it was ever written down. GCSE Computer Science is extremely fond of this fact — expect to be asked about it more than once.